

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION : 1 School Management System

Scenario

A school needs to maintain information about students, classes, and teachers. Each student belongs to a class, and each class has a class teacher. The system should display the student name, class, class teacher, and grade.

Relations

STUDENT

Attribute	Description
Student_ID	Unique ID of student
Student_Name	Name of student
Class_ID	Class assigned to student
Grade	Student's grade

CLASS

Attribute	Description
Class_ID	Unique class ID
Class_Name	Name of class
Teacher_ID	ID of class teacher

TEACHER

Attribute	Description
Teacher_ID	Unique teacher ID
Teacher_Name	Name of teacher

Relational Schema

STUDENT(Student_ID, Student_Name, Class_ID, Grade)

CLASS(Class_ID, Class_Name, Teacher_ID)

TEACHER(Teacher_ID, Teacher_Name)

Relationships:

STUDENT.Class_ID $\rightarrow$ CLASS.Class_ID
CLASS.Teacher_ID $\rightarrow$ TEACHER.Teacher_ID

SQL Table Creation

```
CREATE TABLE TEACHER (  
    Teacher_ID INT PRIMARY KEY,  
    Teacher_Name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE CLASS (  
    Class_ID INT PRIMARY KEY,  
    Class_Name VARCHAR(50) NOT NULL,  
    Teacher_ID INT,  
    FOREIGN KEY (Teacher_ID) REFERENCES TEACHER(Teacher_ID)  
);
```

```
CREATE TABLE STUDENT (  
    Student_ID INT PRIMARY KEY,  
    Student_Name VARCHAR(100) NOT NULL,  
    Class_ID INT,  
    Grade VARCHAR(10),  
    FOREIGN KEY (Class_ID) REFERENCES CLASS(Class_ID)  
);
```

SQL Query

```
SELECT  
    s.Student_ID,  
    s.Student_Name,  
    c.Class_Name,  
    t.Teacher_Name AS Class_Teacher,  
    s.Grade  
FROM STUDENT s  
JOIN CLASS c  
    ON s.Class_ID = c.Class_ID  
JOIN TEACHER t  
    ON c.Teacher_ID = t.Teacher_ID;
```

Explanation

The query uses two JOIN operations:

1. STUDENT is joined with CLASS using Class_ID.
2. CLASS is joined with TEACHER using Teacher_ID.
3. The result displays every student's ID, name, class, class teacher, and grade.

QUESTION : 2 Hospital Appointment System

Scenario

A hospital wants to maintain doctors, patients, and appointments. The hospital needs to identify doctors who have more than 10 appointments during the current month.

Relational Schema

DOCTOR(Doctor_ID, Doctor_Name, Specialization)

PATIENT(Patient_ID, Patient_Name, Phone)

APPOINTMENT(
 Appointment_ID,
 Doctor_ID,
 Patient_ID,
 Appointment_Date,
 Appointment_Time,
 Status
)

Relationships

APPOINTMENT.Doctor_ID → DOCTOR.Doctor_ID

APPOINTMENT.Patient_ID → PATIENT.Patient_ID

Create Tables

```
CREATE TABLE DOCTOR (  
    Doctor_ID INT PRIMARY KEY,  
    Doctor_Name VARCHAR(100) NOT NULL,  
    Specialization VARCHAR(100)  
);
```

```
CREATE TABLE PATIENT (  
    Patient_ID INT PRIMARY KEY,  
    Patient_Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15)  
);
```

```
CREATE TABLE APPOINTMENT (  
    Appointment_ID INT PRIMARY KEY,  
    Doctor_ID INT,  
    Patient_ID INT,  
    Appointment_Date DATE,  
    Appointment_Time TIME,  
    Status VARCHAR(20),  
    FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(Doctor_ID),  
    FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)  
);
```

SQL Query

For MySQL:

```
SELECT  
    d.Doctor_ID,
```

```

    d.Doctor_Name,
    d.Specialization,
    COUNT(a.Appointment_ID) AS Total_Appointments
FROM DOCTOR d
JOIN APPOINTMENT a
    ON d.Doctor_ID = a.Doctor_ID
WHERE YEAR(a.Appointment_Date) = YEAR(CURDATE())
    AND MONTH(a.Appointment_Date) = MONTH(CURDATE())
GROUP BY
    d.Doctor_ID,
    d.Doctor_Name,
    d.Specialization
HAVING COUNT(a.Appointment_ID) > 10;

```

Explanation

- YEAR(CURDATE()) obtains the current year.
- MONTH(CURDATE()) obtains the current month.
- COUNT() calculates the number of appointments.
- GROUP BY creates a group for each doctor.
- HAVING COUNT(...) > 10 selects only doctors having more than 10 appointments.

3. E-Commerce System – Products Never Ordered

Scenario

An e-commerce company maintains product and order information. The objective is to find products that have **never been ordered**.

Relations

PRODUCT(Product_ID, Product_Name, Category, Price)

ORDER_TABLE(Order_ID, Customer_ID, Order_Date)

ORDER_ITEM(Order_ID, Product_ID, Quantity)

Relationship

ORDER_ITEM.Order_ID → ORDER_TABLE.Order_ID

ORDER_ITEM.Product_ID → PRODUCT.Product_ID

Method 1 – Using NOT EXISTS

```

SELECT
    p.Product_ID,
    p.Product_Name,
    p.Category,

```

```
p.Price
FROM PRODUCT p
WHERE NOT EXISTS (
  SELECT 1
  FROM ORDER_ITEM oi
  WHERE oi.Product_ID = p.Product_ID
);
```

Explanation

The subquery searches for an order item corresponding to each product.

If no matching order is found, NOT EXISTS becomes true and the product is displayed.

Method 2 – Using NOT IN

```
SELECT
  Product_ID,
  Product_Name,
  Category,
  Price
FROM PRODUCT
WHERE Product_ID NOT IN (
  SELECT Product_ID
  FROM ORDER_ITEM
);
```

Important Point

NOT EXISTS is generally safer when the subquery column may contain NULL values.

4. University Database – Relational Algebra

Scenario

A university has students and courses. We need to find students who are enrolled in **both DBMS and OS**.

Relations

STUDENT(SID, Student_Name, Department)

COURSE(CID, Course_Name)

ENROLLMENT(SID, CID)

Example:

COURSE

CID Course_Name
C01 DBMS
C02 OS
C03 AI

Step 1 – Find students enrolled in DBMS

DBMS_STUDENTS =
 π SID
(
 σ Course_Name = 'DBMS'
 (ENROLLMENT $\bowtie$ COURSE)
)

Step 2 – Find students enrolled in OS

OS_STUDENTS =
 π SID
(
 σ Course_Name = 'OS'
 (ENROLLMENT $\bowtie$ COURSE)
)

Step 3 – Find students in both

Use **intersection**:

RESULT =
DBMS_STUDENTS $\cap$ OS_STUDENTS

Complete Expression

π SID
(
 σ Course_Name='DBMS'
 (ENROLLMENT $\bowtie$ COURSE)
)
 $\cap$
 π SID
(
 σ Course_Name='OS'
 (ENROLLMENT $\bowtie$ COURSE)
)

To obtain student names:

π Student_Name
(

```

STUDENT ∞
(
  DBMS_STUDENTS ∩ OS_STUDENTS
)
)

```

Explanation

The intersection operation returns only those student IDs that occur in both sets:

DBMS students $\cap$ OS students

Therefore, the result contains students who are enrolled in both courses.

QUESTION : 5 Payroll System – View and Salary Update

Scenario

A company maintains employee salary information. Management wants a view showing employees whose salary is below their department's average salary. Their salary should then be increased by 10%.

Relations

DEPARTMENT(Dept_ID, Dept_Name)

```

EMPLOYEE(
  Emp_ID,
  Emp_Name,
  Dept_ID,
  Salary
)

```

Step 1 – Create the View

```

CREATE VIEW Employees_Below_Dept_Average AS
SELECT
  e.Emp_ID,
  e.Emp_Name,
  e.Dept_ID,
  e.Salary
FROM EMPLOYEE e
WHERE e.Salary < (
  SELECT AVG(e2.Salary)
  FROM EMPLOYEE e2
  WHERE e2.Dept_ID = e.Dept_ID
);

```

Explanation

This is a correlated subquery.

For every employee:

1. The employee's department is identified.
2. The average salary of that department is calculated.
3. The employee's salary is compared with the department average.
4. Employees earning less than the average are included in the view.

Display View

```
SELECT *  
FROM Employees_Below_Dept_Average;
```

Sample Output

Emp_ID	Emp_Name	Dept_ID	Salary
101	Arun	10	28000
105	Priya	10	30000
109	Karthik	20	35000

Step 2 – Update Salary by 10%

```
UPDATE EMPLOYEE e  
SET Salary = Salary * 1.10  
WHERE Salary < (  
    SELECT AVG(e2.Salary)  
    FROM EMPLOYEE e2  
    WHERE e2.Dept_ID = e.Dept_ID  
);
```

Important Note for MySQL

If MySQL reports an error related to updating a table while selecting from the same table, a derived-table approach can be used:

```
UPDATE EMPLOYEE e  
JOIN (  
    SELECT  
        Emp_ID,  
        Salary * 1.10 AS New_Salary  
    FROM EMPLOYEE e1  
    WHERE Salary < (  
        SELECT AVG(e2.Salary)  
        FROM EMPLOYEE e2  
        WHERE e2.Dept_ID = e1.Dept_ID  
    )  
)
```

```
) x
ON e.Emp_ID = x.Emp_ID
SET e.Salary = x.New_Salary;
```

QUESTION : 6 Banking System – JOIN Operations

Scenario

A bank maintains information about customers, accounts, transactions, and branches. The system should retrieve:

- Customer details
- Account details
- Transaction history
- Branch information

Relations

```
CUSTOMER(
    Customer_ID,
    Customer_Name,
    Phone
)
```

```
ACCOUNT(
    Account_No,
    Customer_ID,
    Branch_ID,
    Account_Type,
    Balance
)
```

```
TRANSACTION(
    Transaction_ID,
    Account_No,
    Transaction_Date,
    Transaction_Type,
    Amount
)
```

```
BRANCH(
    Branch_ID,
    Branch_Name,
    City
)
```

Relationships

ACCOUNT.Customer_ID → CUSTOMER.Customer_ID

ACCOUNT.Branch_ID → BRANCH.Branch_ID

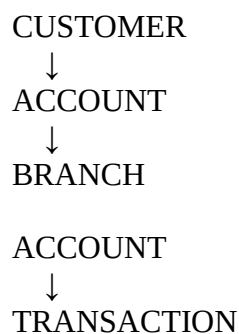
TRANSACTION.Account_No → ACCOUNT.Account_No

SQL Using JOIN

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    a.Account_No,
    a.Account_Type,
    a.Balance,
    t.Transaction_ID,
    t.Transaction_Date,
    t.Transaction_Type,
    t.Amount,
    b.Branch_ID,
    b.Branch_Name,
    b.City
FROM CUSTOMER c
JOIN ACCOUNT a
    ON c.Customer_ID = a.Customer_ID
JOIN BRANCH b
    ON a.Branch_ID = b.Branch_ID
JOIN TRANSACTION t
    ON a.Account_No = t.Account_No;
```

Explanation

The query connects four tables.



The output provides complete banking information in a single result.

Sample Output

Customer	Account	Balance	Transaction	Amount	Branch
Arun	10001	45000	Deposit	10000	Chennai Main
Priya	10002	65000	Withdrawal	5000	Anna Nagar
Karthik	10003	30000	Deposit	15000	T Nagar

Using LEFT JOIN

If we want to display customers even when they have no transactions:

```
SELECT
  c.Customer_Name,
  a.Account_No,
  b.Branch_Name,
  t.Transaction_Date,
  t.Amount
FROM CUSTOMER c
LEFT JOIN ACCOUNT a
  ON c.Customer_ID = a.Customer_ID
LEFT JOIN BRANCH b
  ON a.Branch_ID = b.Branch_ID
LEFT JOIN TRANSACTION t
  ON a.Account_No = t.Account_No;
```

QUESTION : 7 Retail Store – Daily Sales Summary

Scenario

A retail store wants to calculate total sales for each product category every day. Only categories having total sales greater than **Rs. 10,000** should be displayed.

Relation

```
SALES(
  Sale_ID,
  Sale_Date,
  Product_ID,
  Category,
  Quantity,
  Unit_Price
)
```

SQL Query

```
SELECT
  Sale_Date,
```

```

    Category,
    SUM(Quantity * Unit_Price) AS Total_Sales
FROM SALES
GROUP BY
    Sale_Date,
    Category
HAVING SUM(Quantity * Unit_Price) > 10000
ORDER BY
    Sale_Date,
    Total_Sales DESC;

```

Explanation

The expression:

Quantity * Unit_Price

calculates the value of each sale.

SUM() calculates the total sales.

GROUP BY groups records according to:

Sale_Date + Category

HAVING filters groups after aggregation.

Only categories with:

Total Sales > Rs. 10,000

are displayed.

Sample Output

Sale_Date	Category	Total_Sales
2026-08-01	Electronics	25,500
2026-08-01	Clothing	15,800
2026-08-02	Electronics	32,400
2026-08-02	Grocery	12,600

Difference Between WHERE and HAVING

WHERE → filters individual rows

HAVING → filters groups after GROUP BY

Therefore, HAVING is appropriate for filtering based on SUM(), COUNT(), AVG(), etc.

8. Library System – Tuple Relational Calculus

Scenario

A library has multiple genres of books. We need to find members who have borrowed books from all genres available in the library.

Relations

MEMBER(Member_ID, Member_Name)

BOOK(Book_ID, Title, Genre)

BORROW(Member_ID, Book_ID, Borrow_Date)

Objective

Find members m such that for every genre in the BOOK relation, the member has borrowed at least one book belonging to that genre.

Tuple Relational Calculus Expression

```
{
m |
MEMBER(m)
AND
 $\forall g ($ 
  GENRE(g)
 $\rightarrow$ 
 $\exists b \exists br ($ 
  BOOK(b)
  AND BORROW(br)
  AND b.Genre = g.Genre
  AND br.Member_ID = m.Member_ID
  AND br.Book_ID = b.Book_ID
 $)$ 
 $)$ 
}
```

Since GENRE may not be stored as a separate relation, we can express the same idea using the BOOK relation:

```
{
m |
MEMBER(m)
AND
 $\forall b1 ($ 
  BOOK(b1)
 $\rightarrow$ 
```



```

    ∃b2 ∃br (
      BOOK(b2)
      AND BORROW(br)
      AND b2.Genre = b1.Genre
      AND br.Member_ID = m.Member_ID
      AND br.Book_ID = b2.Book_ID
    )
  )
}

```

Meaning

For every book genre:

1. Find a book belonging to that genre.
2. Check whether the member has borrowed a book of that genre.
3. If the member has borrowed at least one book from every genre, include that member in the result.

Example

Suppose the available genres are:

Science
Fiction
History
Technology

A member who borrowed:

Science → Book 101
Fiction → Book 205
History → Book 310
Technology → Book 412

will be included.

A member who borrowed only:

Science
Fiction

will not be included.

9. Inventory Management – CHECK and FOREIGN KEY Constraints

Scenario

An inventory system stores product and supplier information. The database must ensure:

- Product price cannot be negative.
- Product quantity cannot be negative.
- Every product must refer to a valid supplier.

Relations

```
SUPPLIER(  
    Supplier_ID,  
    Supplier_Name  
)
```

```
PRODUCT(  
    Product_ID,  
    Product_Name,  
    Price,  
    Quantity,  
    Supplier_ID  
)
```

Creating Supplier Table

```
CREATE TABLE SUPPLIER (  
    Supplier_ID INT PRIMARY KEY,  
    Supplier_Name VARCHAR(100) NOT NULL  
);
```

Creating Product Table

```
CREATE TABLE PRODUCT (  
    Product_ID INT PRIMARY KEY,  
    Product_Name VARCHAR(100) NOT NULL,  
    Price DECIMAL(10,2),  
    Quantity INT,  
    Supplier_ID INT,  
  
    CONSTRAINT chk_price  
        CHECK (Price >= 0),  
  
    CONSTRAINT chk_quantity  
        CHECK (Quantity >= 0),  
  
    CONSTRAINT fk_supplier  
        FOREIGN KEY (Supplier_ID)  
        REFERENCES SUPPLIER(Supplier_ID)  
);
```

CHECK Constraint

The following condition is enforced:

CHECK (Price >= 0)

Therefore:

```
INSERT INTO PRODUCT  
VALUES (101, 'Keyboard', 850, 20, 1);
```

is valid.

But:

```
INSERT INTO PRODUCT  
VALUES (102, 'Mouse', -500, 10, 1);
```

is rejected because the price is negative.

Similarly:

```
INSERT INTO PRODUCT  
VALUES (103, 'Monitor', 15000, -5, 1);
```

is rejected because quantity cannot be negative.

FOREIGN KEY Enforcement

Suppose supplier IDs are:

1
2
3

Then:

```
INSERT INTO PRODUCT  
VALUES (104, 'Printer', 12000, 5, 2);
```

is valid because supplier 2 exists.

But:

```
INSERT INTO PRODUCT  
VALUES (105, 'Scanner', 8000, 3, 99);
```

will be rejected because supplier 99 does not exist.

Purpose of Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies records
CHECK	Restricts values according to a condition
FOREIGN KEY	Maintains referential integrity
NOT NULL	Prevents missing values

QUESTION : 10 HR System – Relational Schema and Materialized Views

Scenario

An HR system needs to manage employees, departments, attendance, and performance. Management wants monthly summaries of:

1. Employee attendance
2. Employee performance

Because these summaries may be queried frequently, **materialized views** can be used to store the results physically for faster access.

Relational Schema

EMPLOYEE

```
EMPLOYEE(  
    Emp_ID,  
    Emp_Name,  
    Dept_ID,  
    Job_Title,  
    Joining_Date  
)
```

DEPARTMENT

```
DEPARTMENT(  
    Dept_ID,  
    Dept_Name  
)
```

ATTENDANCE

```
ATTENDANCE(  
    Attendance_ID,  
    Emp_ID,  
    Attendance_Date,
```

Status
)

PERFORMANCE

PERFORMANCE(
Performance_ID,
Emp_ID,
Review_Date,
Rating,
Comments
)

Relationships

EMPLOYEE.Dept_ID
↓
DEPARTMENT.Dept_ID

ATTENDANCE.Emp_ID
↓
EMPLOYEE.Emp_ID

PERFORMANCE.Emp_ID
↓
EMPLOYEE.Emp_ID

Create Tables

```
CREATE TABLE DEPARTMENT (  
  Dept_ID INT PRIMARY KEY,  
  Dept_Name VARCHAR(100) NOT NULL  
);  
CREATE TABLE EMPLOYEE (  
  Emp_ID INT PRIMARY KEY,  
  Emp_Name VARCHAR(100) NOT NULL,  
  Dept_ID INT,  
  Job_Title VARCHAR(100),  
  Joining_Date DATE,  
  FOREIGN KEY (Dept_ID)  
    REFERENCES DEPARTMENT(Dept_ID)  
);  
CREATE TABLE ATTENDANCE (  
  Attendance_ID INT PRIMARY KEY,  
  Emp_ID INT,  
  Attendance_Date DATE,  
  Status VARCHAR(20),  
  FOREIGN KEY (Emp_ID)  
    REFERENCES EMPLOYEE(Emp_ID)  
);
```

```

CREATE TABLE PERFORMANCE (
    Performance_ID INT PRIMARY KEY,
    Emp_ID INT,
    Review_Date DATE,
    Rating DECIMAL(3,2),
    Comments VARCHAR(255),
    FOREIGN KEY (Emp_ID)
        REFERENCES EMPLOYEE(Emp_ID)
);

```

Materialized View 1 – Monthly Attendance Summary

The exact materialized-view syntax depends on the DBMS. In systems that support materialized views, the idea is:

```

CREATE MATERIALIZED VIEW Monthly_Attendance_Summary AS
SELECT
    Emp_ID,
    EXTRACT(YEAR FROM Attendance_Date) AS Attendance_Year,
    EXTRACT(MONTH FROM Attendance_Date) AS Attendance_Month,

    COUNT(*) AS Total_Days,

    SUM(
        CASE
            WHEN Status = 'Present' THEN 1
            ELSE 0
        END
    ) AS Present_Days,

    SUM(
        CASE
            WHEN Status = 'Absent' THEN 1
            ELSE 0
        END
    ) AS Absent_Days

FROM ATTENDANCE
GROUP BY
    Emp_ID,
    EXTRACT(YEAR FROM Attendance_Date),
    EXTRACT(MONTH FROM Attendance_Date);

```